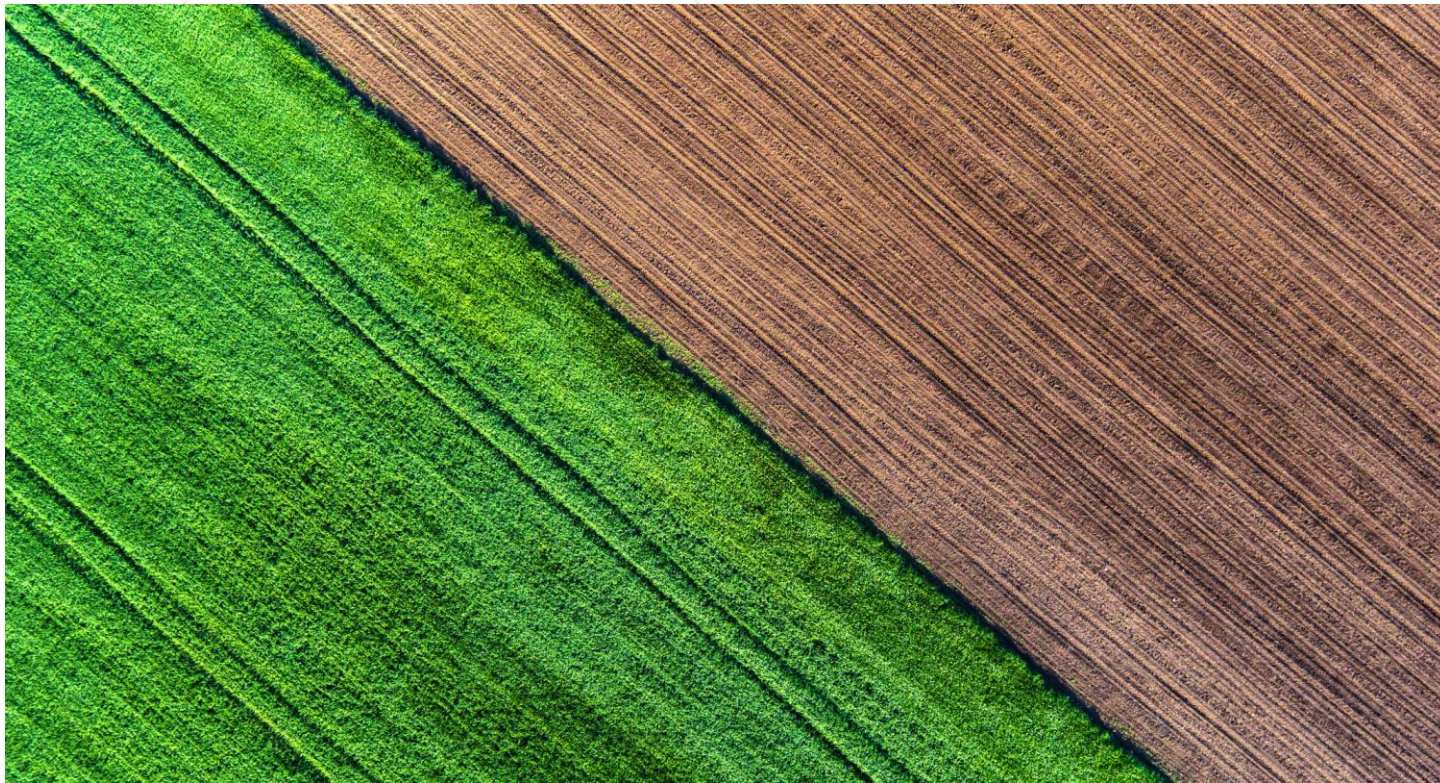




初识进程与线程（下）

北京理工大学计算机学院
金旭亮

Process类

JDK提供了一个Process类型，用于引用本地操作系统创建的进程。

Process提供了许多方法用于获取进程的状态信息，或者操控进程。

创建Process实例，调用ProcessBuilder.start()方法或Runtime.exec()方法。

Process		
m	onExit ()	CompletableFuture <Process >
m	inputReader (Charset)	BufferedReader
m	toHandle ()	ProcessHandle
m	descendants ()	Stream <ProcessHandle >
m	getInputStream ()	InputStream
m	destroy ()	void
m	errorReader (Charset)	BufferedReader
m	pid ()	long
m	isAlive ()	boolean
m	exitValue ()	int
m	getOutputStream ()	OutputStream
m	info ()	Info
m	supportsNormalTermination ()	boolean
m	inputReader ()	BufferedReader
m	children ()	Stream <ProcessHandle >
m	outputWriter (Charset)	BufferedWriter
m	errorReader ()	BufferedReader
m	outputWriter ()	BufferedWriter
m	waitFor ()	int
m	waitFor (long, TimeUnit)	boolean
m	destroyForcibly ()	Process
m	getErrorStream ()	InputStream

ProcessBuilder类

从类名可以知道，ProcessBuilder用于创建一个进程对象，它对应着一个正在运行的本地进程。

ProcessBuilder提供了相应的方法用于指定要启动的进程的各种信息：比如工作目录、命令行参数、环境变量等等，设置完上述信息之后，调用它的start()方法启动相应进程，并得到一个Process对象引用。

ProcessBuilder		
m	redirectInput (File)	ProcessBuilder
m	start()	Process
m	environment ()	Map <String, String >
m	redirectError (Redirect)	ProcessBuilder
m	redirectError (File)	ProcessBuilder
m	command (String [])	ProcessBuilder
m	redirectOutput ()	Redirect
m	redirectOutput (Redirect)	ProcessBuilder
m	redirectError ()	Redirect
m	redirectOutput (File)	ProcessBuilder
m	redirectErrorStream (boolean)	ProcessBuilder
m	command ()	List <String >
m	start(Redirect [])	Process
m	directory (File)	ProcessBuilder
m	redirectInput (Redirect)	ProcessBuilder
m	directory ()	File
m	command (List <String >)	ProcessBuilder
m	redirectInput ()	Redirect
m	inheritIO ()	ProcessBuilder
m	redirects ()	Redirect []
m	startPipeline (List <ProcessBuilder >)	List <Process >
m	environment (String [])?	ProcessBuilder
m	redirectErrorStream ()	boolean

ProcessHandle

为了便于管理进程，操作系统为每个进程分配了一个唯一的标识，这个标识称为“**进程句柄 (Handle)**”，它实际上引用一个复杂的由操作系统管理的数据结构，这一数据结构中包含有进程相关的信息。

JDK中的ProcessHandle，其实就是对操作系统进程句柄所关联信息的封装，通过它可以获取进程相关的信息（比如查询指定进程的父子进程各是哪个），或者进行特定的操作（比如请求操作系统Kill掉某个进程）。

I ProcessHandle		
m	current()	ProcessHandle
m	children()	Stream<ProcessHandle>
m	pid()	long
m	parent()	Optional<ProcessHandle>
m	of(long)	Optional<ProcessHandle>
m	onExit()	CompletableFuture<ProcessHandle>
m	equals(Object)	boolean
m	compareTo(ProcessHandle)	int
m	hashCode()	int
m	destroyForcibly()	boolean
m	info()	Info
m	isAlive()	boolean
m	descendants()	Stream<ProcessHandle>
m	allProcesses()	Stream<ProcessHandle>
m	supportsNormalTermination()	boolean
m	destroy()	boolean

学习提示



Process、ProcessBuilder和ProcessHandle，是JDK中与进程相关的核心类型，包容诸多的成员。



JDK为这三个类型提供了详尽的注释，请同学们打开JDK源码，仔细阅读一遍这些注释。



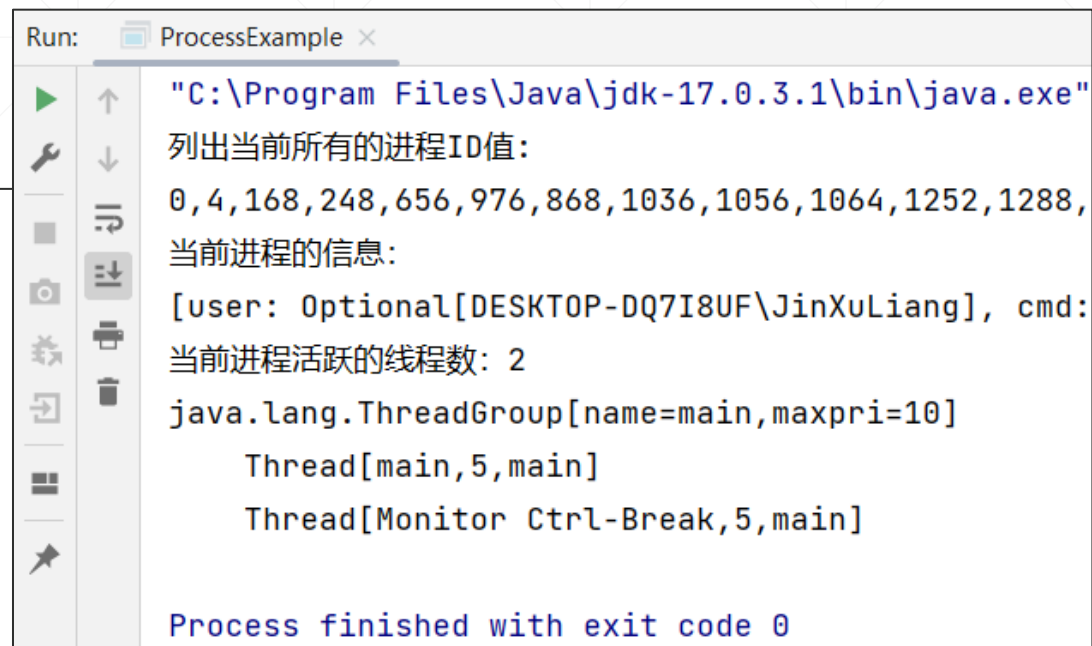
看懂这三个类型的注释，掌握其用法的前提，是学习者必须学过《操作系统原理》这门计算机专业的骨干课程，对操作系统是如何管理进程的有所了解。

Java获取当前进程与线程信息

使用ProcessHandle完成这个任务:

```
private static void getProcessInfo() {  
    System.out.println("列出当前所有的进程ID值:");  
    ProcessHandle.allProcesses()  
        .forEach(p -> System.out.print(p + ","));  
    //获取当前进程的句柄  
    var handle : ProcessHandle = ProcessHandle.current();  
    System.out.println("当前进程ID: " + handle.pid());  
    System.out.println("当前进程的信息: ");  
    System.out.println(handle.info());  
    System.out.println("当前进程活跃的线程数: " + Thread.activeCount());  
    //列举当前线程组中的所有线程  
    Thread.currentThread().getThreadGroup().list();  
}
```

ProcessExample.java



```
Run: ProcessExample x  
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"  
列出当前所有的进程ID值:  
0,4,168,248,656,976,868,1036,1056,1064,1252,1288,  
当前进程的信息:  
[user: Optional[DESKTOP-DQ7I8UF\JinXuLiang], cmd:  
当前进程活跃的线程数: 2  
java.lang.ThreadGroup[name=main,maxpri=10]  
    Thread[main,5,main]  
    Thread[Monitor Ctrl-Break,5,main]  
  
Process finished with exit code 0
```

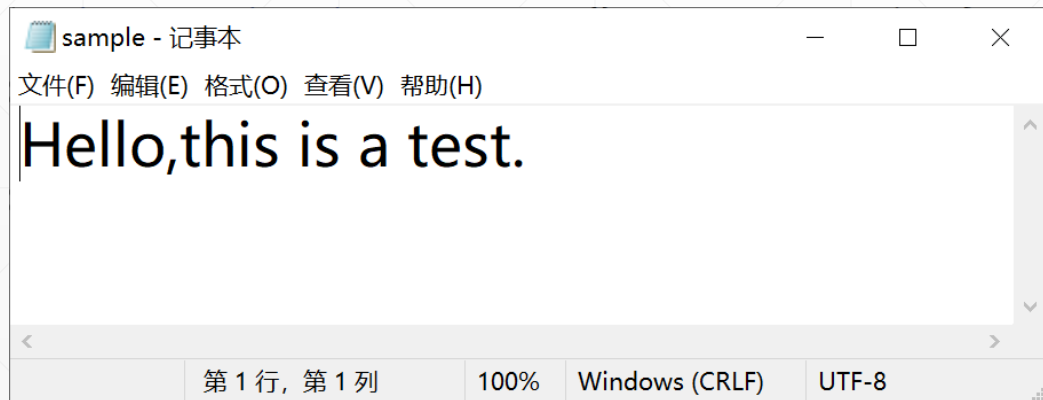
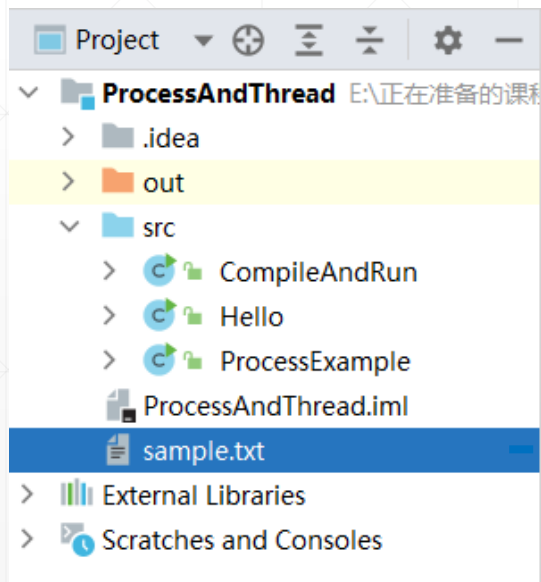
启动记事本进程



使用ProcessBuilder的start()方法，可以启动一个进程：

```
private static void startNotepad() throws IOException {  
    ProcessBuilder builder = new ProcessBuilder(...command: "notepad", "sample.txt");  
    builder.start();  
}
```

ProcessExample.java



截获进程的输出

```
private static void getMyIPInfo() {  
    var processBuilder = new ProcessBuilder(...command: "ipconfig", "/all");  
    try {  
        Process process = processBuilder.start();  
        BufferedReader br = new BufferedReader(  
            new InputStreamReader(process.getInputStream(), charsetName: "GBK"));  
        String line;  
        while ((line = br.readLine()) != null) {  
            System.out.println(line);  
        }  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

调用Windows内置的命令

正在运行的进程的“输出”，对应于Process对象的“输入”



The screenshot shows a Java IDE console window titled "Run: ProcessHandleExample". The command executed is "C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent". The output is the Windows IP configuration details.

```
Run: ProcessHandleExample x  
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent  
Windows IP 配置  
  
主机名 . . . . . : DESKTOP-073T16G  
主 DNS 后缀 . . . . . :  
节点类型 . . . . . : 混合  
IP 路由已启用 . . . . . : 否  
WINS 代理已启用 . . . . . : 否
```


应用实例：运行时动态编译Java源文件



利用ProcessBuilder，我们可以调用javac命令动态编译一个java源代码文件，并截取其编译输出信息。



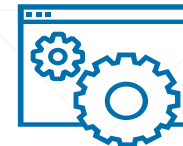
为了能动态地编译，我们需要获取Java的类路径作为javac和java程序的工作路径，可以通过以下方式获取到当前计算机上类路径的值：

```
String workingDir = System.getProperty("java.class.path");  
System.out.println("当前工作目录: " + workingDir);
```



有了这个路径，现在，javac和java就可以顺畅地工作了.....

动态编译与运行Java文件示例



```
public class CompileAndRun {
```

```
    public static void main(String[] args)
```

```
        throws IOException, InterruptedException {
```

```
        String workingDir = System.getProperty("java.class.path");
```

```
        System.out.println("当前工作目录: " + workingDir);
```

```
        //运行时调用javac编译器, 即时编译Java文件, 生成字节码文件: Hello.class
```

```
        compileJavaFile(workingDir, javaFileName: "Hello.java");
```

```
        //运行时调用java命令, 运行Hello.class
```

```
        runJavaClassFile(workingDir, javaClassFileName: "Hello");
```

```
    }
```

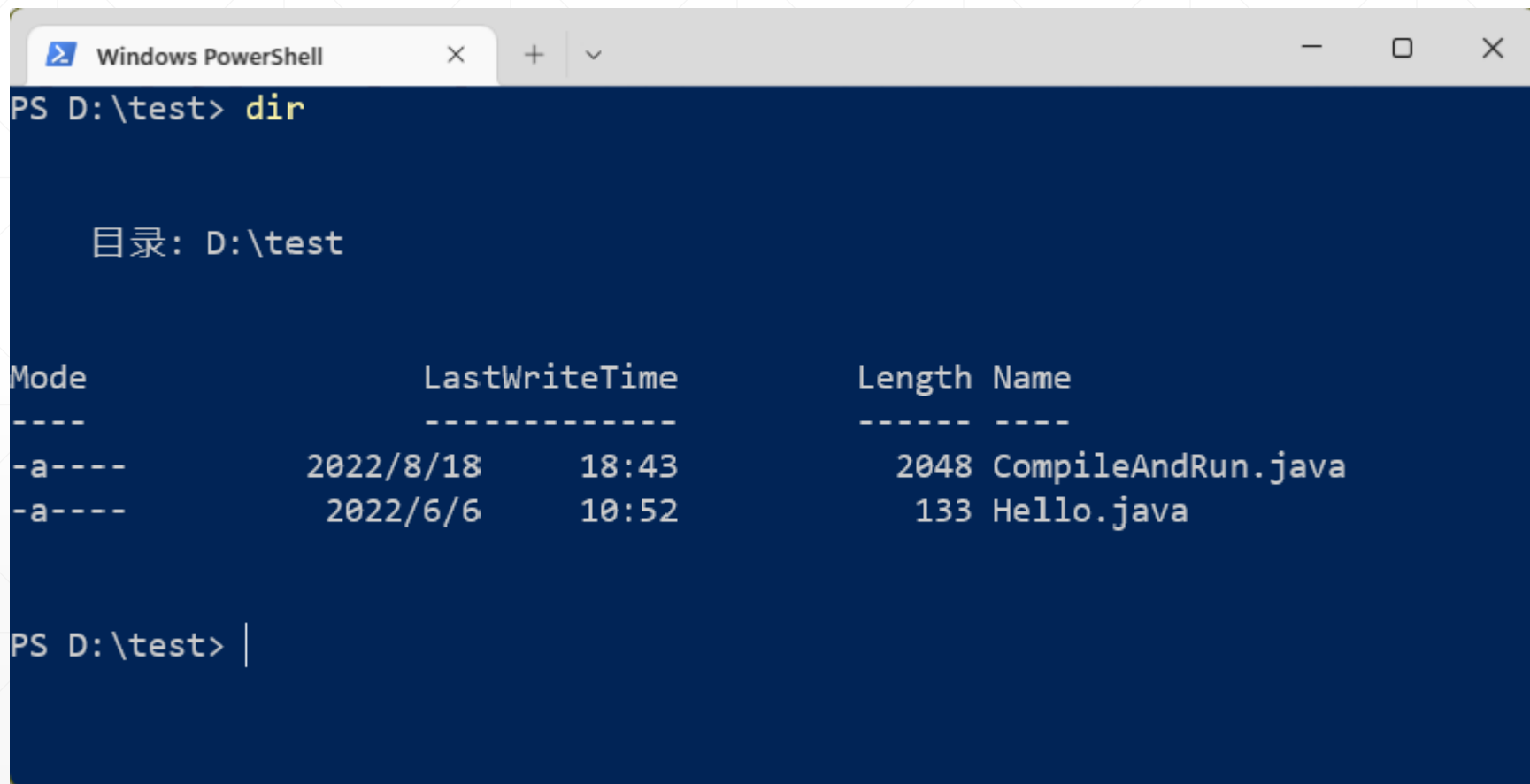
```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("你好, Java! ");  
    }  
}
```

将要被编译的java代码

详细代码后面展开介绍

测试动态编译-1

先将两个Java文件放到某个文件夹（比如D:\test）下：



```
Windows PowerShell
PS D:\test> dir

    目录: D:\test

Mode                LastWriteTime         Length Name
----                -
-a----            2022/8/18      18:43         2048 CompileAndRun.java
-a----            2022/6/6        10:52         133 Hello.java

PS D:\test> |
```

测试动态编译-2

编译主程序（注意要指定文件编码）

```
Windows PowerShell
PS D:\test> javac -encoding UTF-8 CompileAndRun.java
PS D:\test> dir

目录: D:\test

Mode                LastWriteTime         Length Name
----                -
-a----             2022/8/18        2342 CompileAndRun.class
-a----             2022/8/18        2048 CompileAndRun.java
-a----             2022/6/6          133 Hello.java

PS D:\test> |
```

Java文件中有汉字，则必须在编译时指定字符编码，否则，javac会报告错误。

测试动态编译-3

```
Windows PowerShell
PS D:\test> java CompileAndRun
当前工作目录: .
Compiled finished
你好, Java!
PS D:\test> dir

目录: D:\test

Mode                LastWriteTime         Length Name
----                -
-a----            2022/8/18      18:47           2342 CompileAndRun.class
-a----            2022/8/18      18:43           2048 CompileAndRun.java
-a----            2022/8/18      18:50            420 Hello.class
-a----            2022/6/6        10:52            133 Hello.java
```

Hello.java被javac动态地编译为字节码，然后被java命令启动运行。

Hello.java生成的字节码文件。

动态编译示例源码解析-1

动态编译Java文件

```
private static void compileJavaFile(String workingDir, String javaFileName)
    throws InterruptedException, IOException {
    //由于Hello.java中包容有中文字符, 所以, 需要在编译时指定字符编码
    var compiler = new ProcessBuilder(...command: "javac", "-encoding", "UTF-8",
        "-d", workingDir, workingDir + "\\\" + javaFileName);
    //指定编译器工作路径
    compiler.directory(new File(workingDir));
    //等待编译完成
    compiler.start().waitFor();
    System.out.println("Compiled finished");
}
```

waitFor()是Process类
提供的方法

动态编译示例源码解析-2

```
private static void runJavaClassFile(String workingDir, String javaClassName) {
    ProcessBuilder processBuilder =
        new ProcessBuilder( ...command: "java", javaClassName);
    try {
        processBuilder.directory(new File(workingDir));
        Process process = processBuilder.start();
        //捕获进程的输出, 输出在控制台窗口中
        //为了在Windows的命令提示符窗口中正确显示汉字, 按GBK方式解码字符串
        var br = new BufferedReader(
            new InputStreamReader(process.getInputStream(),
                Charset.forName( charsetName: "GBK")));
        //在命令提示符窗口显示示例程序运行时的输出
        String line;
        while ((line = br.readLine()) != null) {
            System.out.println(line);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

注意：此示例仅适用于Windows，在Window11中运行通过。

学以致用



通过本讲示例，你应该知道了如何动态编译一个Java源代码文件。

请在本示例的基础之上，试着编写一个简单的Java IDE，提供一个文本控件让用户输入Java代码，将其保存为.java文件，然后程序调用javac编译器编译它，之后调用java命令运行编译好的字节码文件。